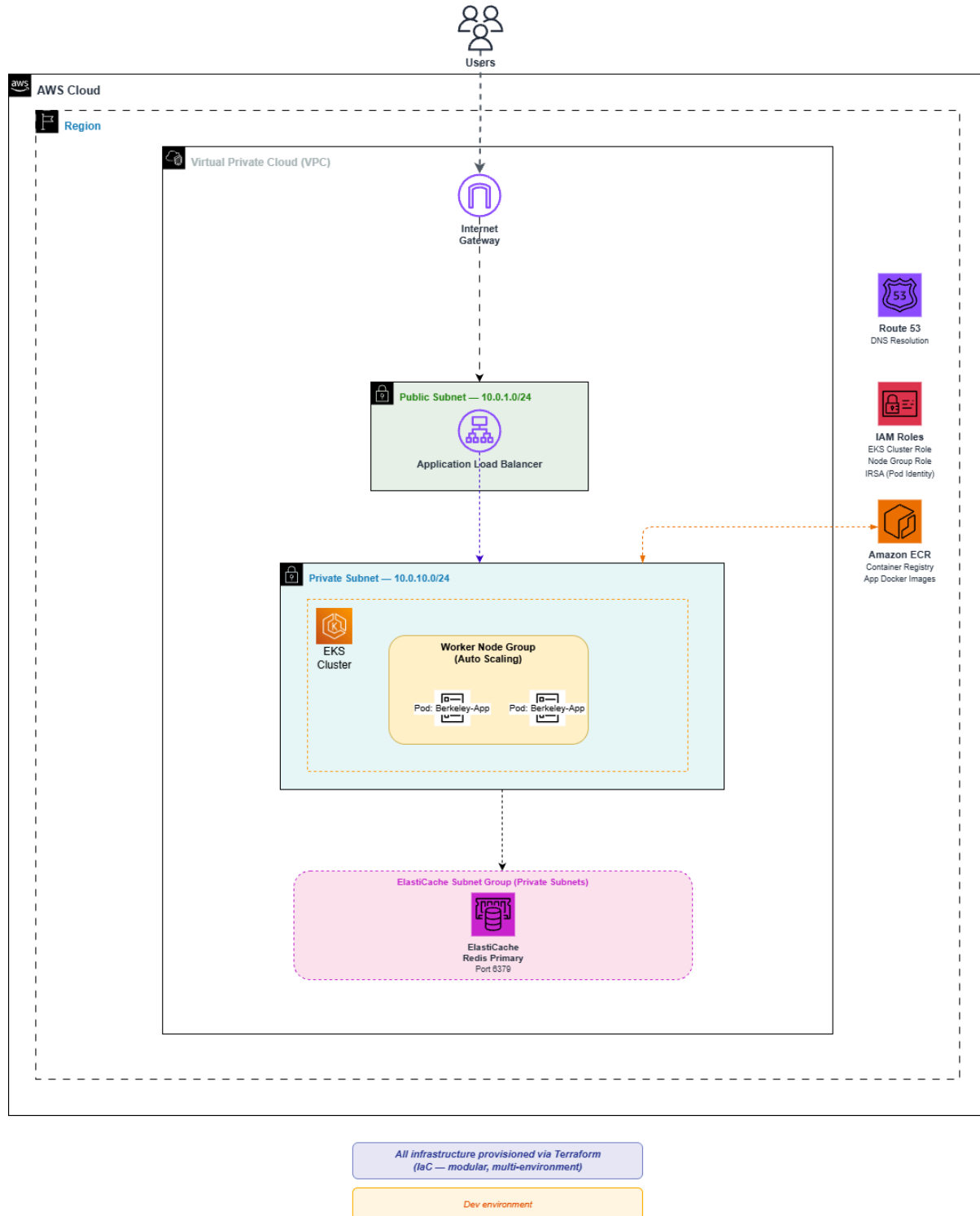


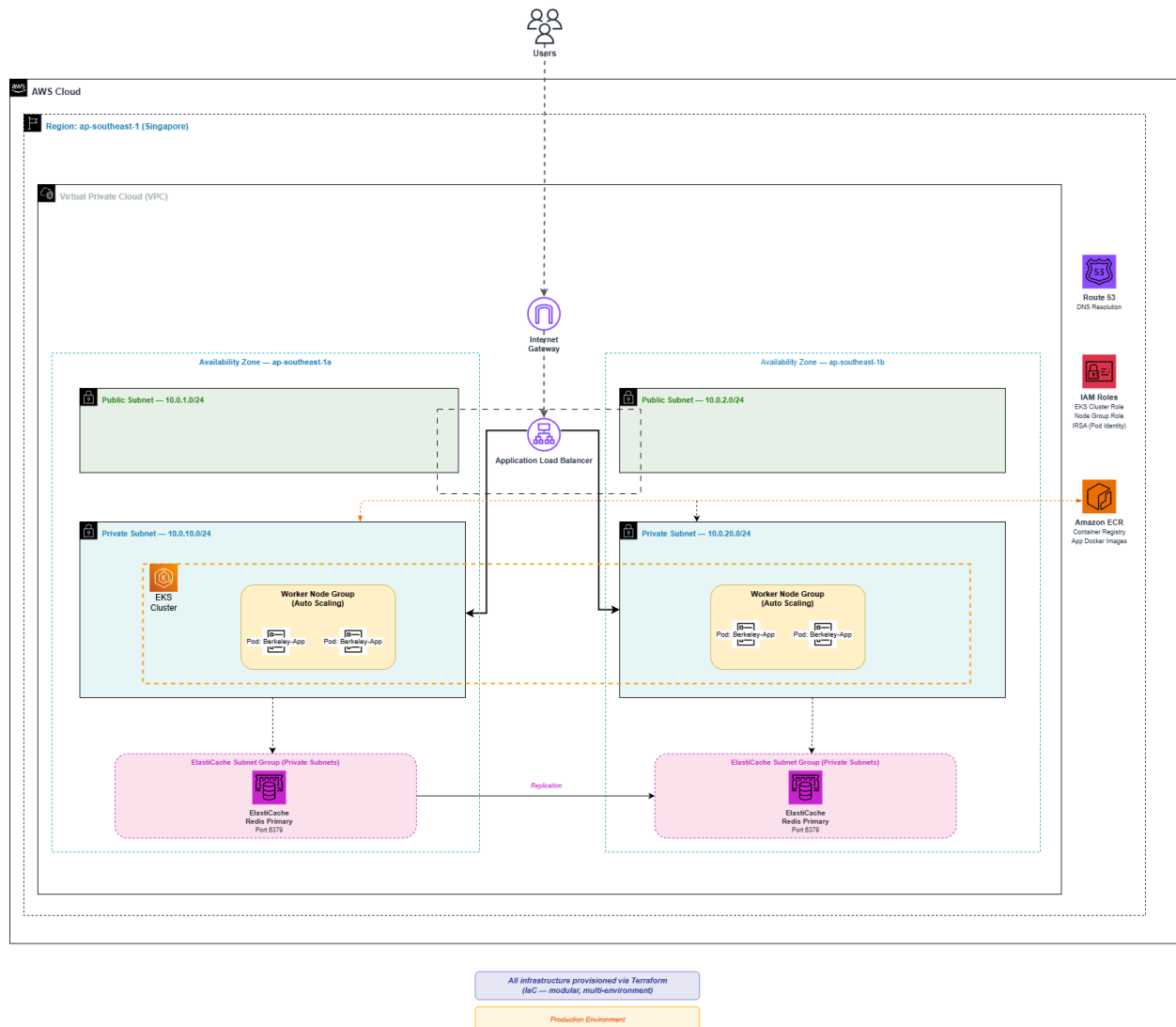
Berkeley Visitor Counter

AWS Architecture Overview

1. Dev Environment Architecture



2. Prod environment Architecture



3. Architecture Overview

This document details the cloud infrastructure that will be provisioned for the Berkeley Visitor Counter application to be deployed on AWS EKS.

Infrastructure Components:

- VPC with public and private subnets across 2 Availability Zones
- EKS cluster with managed node groups
- ElastiCache Redis for application data
- IAM users and roles with least privilege access
- Application Load Balancer for public access
- KMS encryption for secrets
- CloudWatch for logging

All infrastructure will be provisioned via Terraform using official AWS modules. The same code will support multiple environments (dev, qa, nonprod, prod) through separate variable files.

4. Network Design

The VPC will be spread across two Availability Zones for high availability.

Subnets:

- Public Subnet 1 — AZ 1a — ALB
- Public Subnet 2 — AZ 1b — ALB
- Private Subnet 1 — AZ 1a — EKS Workers, ElastiCache Primary
- Private Subnet 2 — AZ 1b — EKS Workers, ElastiCache Replica

Routing:

- Public subnets will route to Internet Gateway for direct internet access

5. EKS Cluster

- API endpoint: Public + Private access
- Logging: api, audit, authenticator will be sent to CloudWatch
- Managed node group with auto scaling
- Dev will use smaller instances with fewer nodes, Prod will use larger instances with higher scaling

6. ElastiCache (Redis)

- Subnet group will span both private subnets
- Security group will allow access only from EKS worker nodes on port 6379
- Encryption at rest and in transit will be enabled
- CloudWatch slow-log will be enabled

Dev: Single node, no replica, Primary + replica across AZs, multi-AZ enabled, automatic failover

Non-Prod : Primary will be in AZ 1a, replica in AZ 1b. If primary fails, replica will auto-promote. Application will connect via primary endpoint which auto-redirects.

Prod: Primary will be in AZ 1a, replica in AZ 1b. If primary fails, replica will auto-promote. Application will connect via primary endpoint which auto-redirects.

7. IAM & Security

Access Model:

- **Cluster manager** — will have least privilege for EKS, VPC, ElastiCache, IAM for EKS
- **Developer role** — will have ReadOnlyAccess on AWS

Security Measures:

- ElastiCache encryption at rest and in transit
- Workers and Redis will be in private subnets only
- Security group will restrict Redis access to EKS nodes only
- No AdministratorAccess on any user
- CloudWatch logging for api, audit, authenticator

8. Terraform Structure

Files:

- *provider.tf* — AWS provider configuration and Terraform version
- *backend.tf* — S3 remote state with encryption
- *variables.tf* — All variable definitions
- *main.tf* — IAM, VPC, EKS, ElastiCache modules
- *outputs.tf* — Resource outputs (cluster endpoint, Redis endpoint, ARNs)
- *dev.tfvars* — Dev environment variable values
- *non-prod.tfvars* — Non-Prod environment variable values
- *prod.tfvars* — Prod environment variable values

Remote state will be stored in an S3 bucket with encryption enabled.

9. Environment Configuration

Each environment will have its own *.tfvars* file containing environment-specific values. The Terraform code will remain the same only the variable values will change per environment.

dev.tfvars : Development environment. Will use smaller, cost-effective instances. Single Redis node with no replication. Suitable for development and testing. Lower node count with moderate auto scaling limits.

non-prod.tfvars : Non-production environment. Will mirror production configuration with same instance types and Redis replication to validate changes before production deployment. Multi-AZ failover will be enabled to test high availability scenarios.

prod.tfvars : Production environment. Will use larger instances for better performance. Redis primary with replica for high availability. Multi-AZ failover will be enabled. Higher node counts with wider auto scaling range to handle production traffic.

Additional *.tfvars* files can be created for other environments following the same pattern. Each file will override variables like instance types, node counts, Redis configuration, and scaling parameters appropriate for that environment.

To deploy a specific environment, the corresponding *.tfvars* file will be passed:

None

```
terraform apply -var-file="dev.tfvars"
terraform apply -var-file="prod.tfvars"
terraform apply -var-file="dev.tfvars"
```

This approach will ensure consistency across environments while allowing each to be sized and configured independently.

10. Terraform Commands

Initialize Terraform:

None

```
terraform init
terraform init -upgrade
```

Validate and Format:

None

```
terraform validate
terraform fmt
```

Plan:

None

```
terraform plan -var-file="dev.tfvars"
terraform plan -var-file="non-prod.tfvars"
terraform plan -var-file="prod.tfvars"
```

Apply:

None

```
terraform apply -var-file="dev.tfvars"  
terraform apply -var-file="prod.tfvars"  
terraform apply -var-file="dev.tfvars"
```